EMPLOYEE DATA CLEANING USING SQL SERVER

SQL Data Cleaning & Transformation

Prepared By: Ritesh Chaudhary

Contents

1. Introduction **2**

2. Project Objectives **2**

3. Tools and Technologies **2**

4. Dataset Description **3**

5. Project Implementation and Methodology **3**

5.1 Database Creation 3

5.2 Creating the Raw/Staging Table 3

5.3 Importing CSV Data 4

5.4 Initial Data Inspection 4

5.5 Duplicate Detection and Removal 4

5.6 Missing Value Detection 5

5.7 Missing Value Treatment 5

5.8 Removing Unwanted Spaces 6

5.9 Standardising Phone Numbers 7

5.10 Creating the Cleaned Table 7

5.11 Data Type Conversion and Insertion 7

5.12 Final Data Validation 8

5.13 Feature Engineering 8

6. Key SQL Concepts and Functions Used **8**

7. Expected Project Outcome **9**

8. Conclusion **9**

1. Introduction

Data cleaning is an important part of data analysis and database management. Raw datasets often contain duplicate records, missing values, unwanted spaces, inconsistent formats, and incorrect data types. These issues can affect the accuracy and reliability of further data analysis.

This project focuses on cleaning and transforming an employee dataset using Microsoft SQL Server. The project uses SQL queries to import raw CSV data, identify data quality issues, handle missing values, remove duplicate records, standardize text and phone number formats, and convert columns into appropriate SQL data types.

The project follows a staging and cleaning approach, where the original CSV data is first imported into a raw table before being processed and transferred into a properly structured table.

2. Project Objectives

The main objectives of this project are:

- Import employee records from a CSV file into SQL Server.
- Create separate raw and cleaned database tables.
- Identify and remove duplicate employee records.
- Detect and handle missing values.
- Remove unwanted leading and trailing spaces.
- Standardize phone number formatting.
- Convert raw text columns into appropriate SQL data types.
- Validate the cleaned dataset.
- Create a derived Full Name column using existing employee information.

3. Tools and Technologies

Tool / Technology	**Purpose**
Microsoft SQL Server	Database management and data processing
SQL Server Management Studio (SSMS)	Writing and executing SQL queries
T-SQL	Data cleaning, transformation and validation
CSV	Source dataset format
Python (Pandas)	Supporting statistical analysis for skewness and comparison of data-cleaning concepts

4. Dataset Description

The project uses an employee dataset stored in a CSV file named Employees.csv.

The dataset contains 11 columns representing employee information.

Column Name	Description	Final SQL Data Type
EMPLOYEE_ID	Unique employee identifier	INT
FIRST_NAME	Employee's first name	VARCHAR(100)
LAST_NAME	Employee's last name	VARCHAR(100)
EMAIL	Employee email identifier	VARCHAR(100)
PHONE_NUMBER	Employee contact number	VARCHAR(50)
HIRE_DATE	Employee hiring date	DATE
JOB_ID	Employee job identifier	VARCHAR(50)
SALARY	Employee salary	DECIMAL(12, 2)
COMMISSION_PCT	Employee commission percentage	DECIMAL(5, 2)
MANAGER_ID	Employee manager identifier	INT
DEPARTMENT_ID	Employee department identifier	INT

The original dataset contains 111 records and 11 columns before data cleaning.

5. Project Implementation and Methodology

The project is divided into the following stages.

5.1 Database Creation

A database named EmployeeDataDB is created in SQL Server to store and process employee records.

CREATE DATABASE EmployeeDataDB;

USE EmployeeDataDB;

5.2 Creating the Raw/Staging Table

A staging table named Employees_Raw is created to store the original CSV data.

All 11 columns are initially defined using the VARCHAR data type. This approach allows raw values to be imported before performing data type conversion and cleaning operations.

It also helps prevent import failures caused by inconsistent values, such as different date formats, blank fields, or numeric values stored as text.

5.3 Importing CSV Data

The BULK INSERT statement is used to import employee records from the CSV file into Employees_Raw.

```
BULK INSERT Employees_Raw
FROM 'C:\Path\Employees.csv'
WITH (
   FORMAT = 'CSV',
   FIRSTROW = 2,
   FIELDTERMINATOR = ',',
   ROWTERMINATOR = '\n',
   TABLOCK
);
```

The import process skips the header row and loads the remaining records into the staging table.

5.4 Initial Data Inspection

After importing the dataset, SQL queries are executed to inspect the imported records and understand the table structure.

The following operations are performed:

- Display all imported records.
- Count the total number of records using COUNT(*).
- Inspect column names and data types through INFORMATION_SCHEMA.COLUMNS.
- Display the first five records using SELECT TOP 5.

These operations help verify the imported dataset before cleaning begins.

5.5 Duplicate Detection and Removal

Duplicate employee records are identified using GROUP BY and HAVING COUNT(*) > 1, based on the EMPLOYEE_ID column.

The ROW_NUMBER() window function is then used with a Common Table Expression (CTE) to identify duplicate records.

The first occurrence is retained, while subsequent records with the same employee ID are identified for removal.

```
WITH DuplicateCTE AS (
   SELECT *,
      ROW_NUMBER() OVER (
         PARTITION BY EMPLOYEE_ID
         ORDER BY EMPLOYEE_ID
      ) AS rn
   FROM Employees_Raw
)
DELETE FROM DuplicateCTE
WHERE rn > 1;
```

After deletion, duplicate employee IDs are checked again to verify the result.

5.6 Missing Value Detection

Missing values are identified across all 11 columns.

The SUM(CASE WHEN ... THEN 1 ELSE 0 END) technique is used to count NULL and empty-string values in each column.

The inspection identifies missing values in the following columns:

- COMMISSION_PCT
- MANAGER_ID
- DEPARTMENT_ID

The missing-value counts and percentages are considered before applying suitable replacement methods.

5.7 Missing Value Treatment

Different approaches are applied depending on the nature of the columns.

A. Commission Percentage

The COMMISSION_PCT column is treated as numerical data.

The missing-value percentage is calculated as:

72 / 107 × 100 = 67.29%

Since the missing percentage is below the project's considered 70–75% threshold, the column is retained.

Python Pandas is used to calculate the skewness:

df['COMMISSION_PCT'].skew()

The calculated skewness is approximately 0.0966.

Based on the project's selected statistical rule, the mean is used for replacement.

The SQL AVG() function calculates the mean commission percentage, which is approximately 0.22.

The UPDATE statement then replaces missing commission values with 0.22.

B. Manager ID

The MANAGER_ID column is treated as categorical identifier data.

The most frequently occurring value (mode) is identified using GROUP BY, COUNT(*), and ORDER BY.

The calculated mode is 100.

Missing manager IDs are replaced with 100.

C. Department ID

Similarly, the mode of the DEPARTMENT_ID column is identified.

The calculated mode is 50.

Missing department IDs are replaced with 50.

Finally, SQL queries are executed again to verify that the targeted columns no longer contain NULL or empty values.

5.8 Removing Unwanted Spaces

The TRIM() function is used to identify and remove unwanted leading and trailing spaces from text columns.

The following columns are included:

- FIRST_NAME
- LAST_NAME
- EMAIL
- JOB_ID

Original and trimmed values are compared using SELECT queries before applying the cleaning operation.

This helps maintain consistency in employee names and other text-based information.

5.9 Standardizing Phone Numbers

The REPLACE() function is used to standardize phone number formatting.

In the original dataset, phone numbers contain periods as separators.

The periods are replaced with hyphens to create a consistent format.

For example:

Original: 515.123.4567

Cleaned: 515-123-4567

This improves consistency when displaying or working with contact information.

5.10 Creating the Cleaned Table

A separate table named Employees_Clean is created with appropriate SQL data types.

Unlike the raw staging table, the cleaned table uses INT, DATE, DECIMAL, and VARCHAR according to the nature of each column.

This provides a properly structured destination for the transformed employee records.

5.11 Data Type Conversion and Insertion

The INSERT INTO ... SELECT statement transfers the processed data from Employees_Raw into Employees_Clean.

The following SQL functions are used:

- TRIM() removes unwanted spaces from text fields.
- TRY_CONVERT() converts text values into the required SQL data types.
- INSERT INTO stores the transformed records in the cleaned table.

TRY_CONVERT() is used instead of CAST() because invalid values return NULL rather than immediately terminating the conversion query with an error.

The resulting table contains properly typed employee records suitable for further database operations and analysis.

5.12 Final Data Validation

After inserting the records, several validation queries are executed to inspect the cleaned dataset.

The validation process includes:

1. Checking the final table structure through INFORMATION_SCHEMA.COLUMNS.
2. Displaying the cleaned employee records.
3. Counting the total number of records.
4. Checking NULL values across all 11 columns.
5. Checking duplicate employee IDs again.

These checks help determine whether the cleaning and transformation operations have produced the expected output.

5.13 Feature Engineering

The final stage demonstrates basic feature engineering by combining the employee's first and last names into a new derived column called FULL_NAME.

The SQL CONCAT() function is used for this operation.

```
SELECT
   EMPLOYEE_ID,
   FIRST_NAME,
   LAST_NAME,
   CONCAT(FIRST_NAME, ' ', LAST_NAME) AS FULL_NAME
FROM Employees_Clean;
```

This produces a full employee name from the existing columns without modifying the original table structure.

6. Key SQL Concepts and Functions Used

The project demonstrates practical use of the following SQL concepts:

SQL Concept / Function	Application
CREATE DATABASE	Creating the project database
CREATE TABLE	Creating raw and cleaned tables
BULK INSERT	Importing CSV records
SELECT	Retrieving employee records

COUNT()	Counting records
GROUP BY	Grouping employee records
HAVING	Identifying duplicate records
ROW_NUMBER()	Identifying duplicate rows
CTE	Organizing duplicate identification and deletion
CASE WHEN	Counting missing values
AVG()	Calculating the commission mean
ORDER BY	Identifying the most frequent values
UPDATE	Replacing missing values and modifying data
TRIM()	Removing unwanted spaces
REPLACE()	Standardizing phone number formatting
TRY CONVERT()	Converting raw text into appropriate data types
INFORMATION_SCHEMA	Inspecting table structure
CONCAT()	Creating the derived full name

7. Expected Project Outcome

After completing the implemented operations, the project produces a cleaned employee dataset stored in the Employees_Clean table.

The expected outcomes include:

- Employee records imported into SQL Server.
- Duplicate employee IDs identified and additional duplicate records removed.
- Missing values in the three targeted columns handled using selected statistical methods.
- Unwanted leading and trailing spaces addressed.
- Phone number formatting standardized.
- Raw text values converted into appropriate SQL data types.
- Cleaned records validated through SQL queries.
- A derived FULL_NAME field generated using existing employee information.

The final cleaned table provides a structured dataset that can be used for further SQL queries, reporting, and data analysis.

8. Conclusion

This project demonstrates a practical approach to employee data cleaning and transformation using Microsoft SQL Server.

Through this implementation, I applied SQL concepts such as data importing, duplicate detection, missing-value treatment, string manipulation, data type conversion, Common Table Expressions, window functions, and data validation.

The project also demonstrates how SQL Server and Python Pandas can be used together for data-cleaning tasks, particularly when statistical analysis is required.

Overall, this assignment helped develop practical knowledge of SQL-based data preparation and database operations, which are important skills for working with real-world datasets in data analytics and software development.

End of Documentation